Name: Saloni P Pawar

Roll no: 45

Sec: C

```
import pandas as pd
import numpy as np

import os

os.getcwd()

'C:\\Users\\Kabir\\Music'

os.chdir(r"C:\\Users\\kabir\\Desktop")

data=pd.read_csv("heart.csv")

data.head()

   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak
slope  \
0   52    1   0       125   212    0        1      168      0      1.0
2
1   53    1   0       140   203    1        0      155      1      3.1
0
2   70    1   0       145   174    0        1      125      1      2.6
0
3   61    1   0       148   203    0        1      161      0      0.0
2
4   62    0   0       138   294    1        1      106      0      1.9
1

   ca  thal  target
0   2     3       0
1   0     3       0
2   0     3       0
3   1     3       0
4   3     2       0

data.tail()

      age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang
oldpeak  \
1020   59    1   1       140   221    0        1      164      1
```

```
0.0
1021   60    1   0       125   258    0        0      141      1
2.8
1022   47    1   0       110   275    0        0      118      1
1.0
1023   50    0   0       110   254    0        0      159      0
0.0
1024   54    1   0       120   188    0        1      113      0
1.4

      slope  ca  thal  target
1020      2   0     2       1
1021      1   1     3       0
1022      1   1     2       0
1023      2   0     2       1
1024      1   1     3       0

data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       1025 non-null   int64
 1   sex       1025 non-null   int64
 2   cp        1025 non-null   int64
 3   trestbps  1025 non-null   int64
 4   chol      1025 non-null   int64
 5   fbs       1025 non-null   int64
 6   restecg   1025 non-null   int64
 7   thalach   1025 non-null   int64
 8   exang     1025 non-null   int64
 9   oldpeak   1025 non-null   float64
 10  slope     1025 non-null   int64
 11  ca        1025 non-null   int64
 12  thal      1025 non-null   int64
 13  target    1025 non-null   int64
dtypes: float64(1), int64(13)
memory usage: 112.2 KB

data.describe()

               age          sex           cp     trestbps        chol
\
count  1025.000000  1025.000000  1025.000000  1025.000000  1025.00000

mean     54.434146     0.695610     0.942439   131.611707   246.00000

std       9.072290     0.460373     1.029641    17.516718    51.59251
```

```
min       29.000000     0.000000     0.000000    94.000000   126.00000

25%       48.000000     0.000000     0.000000   120.000000   211.00000

50%       56.000000     1.000000     1.000000   130.000000   240.00000

75%       61.000000     1.000000     2.000000   140.000000   275.00000

max       77.000000     1.000000     3.000000   200.000000   564.00000


                fbs      restecg      thalach        exang      oldpeak
\
count  1025.000000  1025.000000  1025.000000  1025.000000  1025.000000

mean      0.149268     0.529756   149.114146     0.336585     1.071512

std       0.356527     0.527878    23.005724     0.472772     1.175053

min       0.000000     0.000000    71.000000     0.000000     0.000000

25%       0.000000     0.000000   132.000000     0.000000     0.000000

50%       0.000000     1.000000   152.000000     0.000000     0.800000

75%       0.000000     1.000000   166.000000     1.000000     1.800000

max       1.000000     2.000000   202.000000     1.000000     6.200000


              slope           ca         thal       target
count  1025.000000  1025.000000  1025.000000  1025.000000
mean      1.385366     0.754146     2.323902     0.513171
std       0.617755     1.030798     0.620660     0.500070
min       0.000000     0.000000     0.000000     0.000000
25%       1.000000     0.000000     2.000000     0.000000
50%       1.000000     0.000000     2.000000     1.000000
75%       2.000000     1.000000     3.000000     1.000000
max       2.000000     4.000000     3.000000     1.000000

data.shape

(1025, 14)

data.size

14350

data.ndim

2
```

```
data.isna()
```

```
        age    sex     cp  trestbps   chol    fbs  restecg  thalach
exang  \
0     False  False  False     False  False  False    False    False
False
1     False  False  False     False  False  False    False    False
False
2     False  False  False     False  False  False    False    False
False
3     False  False  False     False  False  False    False    False
False
4     False  False  False     False  False  False    False    False
False
...     ...    ...    ...       ...    ...    ...      ...      ...
...
1020  False  False  False     False  False  False    False    False
False
1021  False  False  False     False  False  False    False    False
False
1022  False  False  False     False  False  False    False    False
False
1023  False  False  False     False  False  False    False    False
False
1024  False  False  False     False  False  False    False    False
False

      oldpeak  slope     ca   thal  target
0       False  False  False  False   False
1       False  False  False  False   False
2       False  False  False  False   False
3       False  False  False  False   False
4       False  False  False  False   False
...       ...    ...    ...    ...     ...
1020    False  False  False  False   False
1021    False  False  False  False   False
1022    False  False  False  False   False
1023    False  False  False  False   False
1024    False  False  False  False   False

[1025 rows x 14 columns]
```

```
data.isna().any()
```

```
age         False
sex         False
cp          False
trestbps    False
chol        False
fbs         False
```

```
restecg     False
thalach     False
exang       False
oldpeak     False
slope       False
ca          False
thal        False
target      False
dtype: bool
```

```
data.isna().sum()
```

```
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          0
thal        0
target      0
dtype: int64
```

Independent and Dependent Variables

```
x=data.drop("target", axis=1)
y=data["target"]
```

Splitting of DataSet into train and Test¶

```
#splitting the data into training and testing data sets
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2 ,rand
om_state=42)
```

Logistic Regression

```
from sklearn.linear_model import LogisticRegression
```

```
log = LogisticRegression(max_iter=1000)
log.fit(x_train, y_train)
```

```
LogisticRegression(max_iter=1000)
```

```
y_pred1 = log.predict(x_test)
```

```
from sklearn.metrics import accuracy_score
```

```
accuracy_score (y_test,y_pred1)
```

```
0.7951219512195122
```

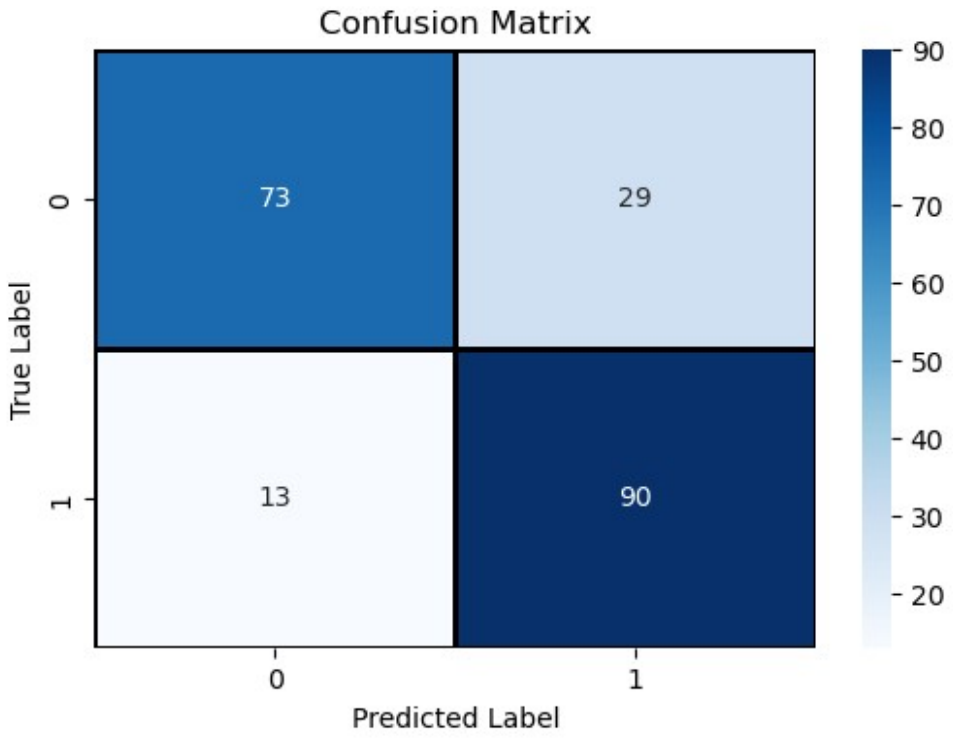
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred1)

labels = np.unique(y_test)  # Get unique class labels
cm_df = pd.DataFrame(cm, index=labels, columns=labels)

# Plot confusion matrix using seaborn
plt.figure(figsize=(6, 4))
sns.heatmap(cm_df, annot=True, fmt='d', cmap='Blues', linewidths=1,
linecolor='black')

plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix")
plt.show()
```



KNN Classifier

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

knn=KNeighborsClassifier()

knn.fit(x_train, y_train)

KNeighborsClassifier()

y_pred2=knn.predict(x_test)

accuracy = accuracy_score(y_test, y_pred2)

accuracy

0.7317073170731707
```

Support Vector Classifier / Machine (SVC/SVM)

```
from sklearn import svm
svm=svm.SVC()
svm.fit(x_train, y_train)
```

```
SVC()
y_pred3=svm.predict(x_test)
accuracy_score (y_test,y_pred3)
0.6829268292682927
```

Decision Trees Algorithm

```
from sklearn.tree import DecisionTreeClassifier
dt=DecisionTreeClassifier()
dt.fit(x_train, y_train)
DecisionTreeClassifier()
y_pred4=dt.predict(x_test)
accuracy_score (y_test,y_pred4)
0.9853658536585366
```

Random Forest Classifier

```
from sklearn.ensemble import RandomForestClassifier
rf=RandomForestClassifier()
rf.fit(x_train, y_train)
RandomForestClassifier()
y_pred5=rf.predict(x_test)
accuracy_score (y_test,y_pred5)
0.9853658536585366
```